

GitHub Repository: <https://github.com/dylgibs/GibsonCS449PegSolitaireProject>

Sprint #0 Report

Instructions

Objectives

- Make decisions on the Peg Solitaire software development project.
- Learn unit testing and GUI programming in the language of your choice.

Deliverables and Grading Policy

Read the “CS 449 Homework Overview” document **carefully** and make the key decisions for the software development. Use the following template to complete your report.

1. Key Decisions of the Solitaire Project (2 points)

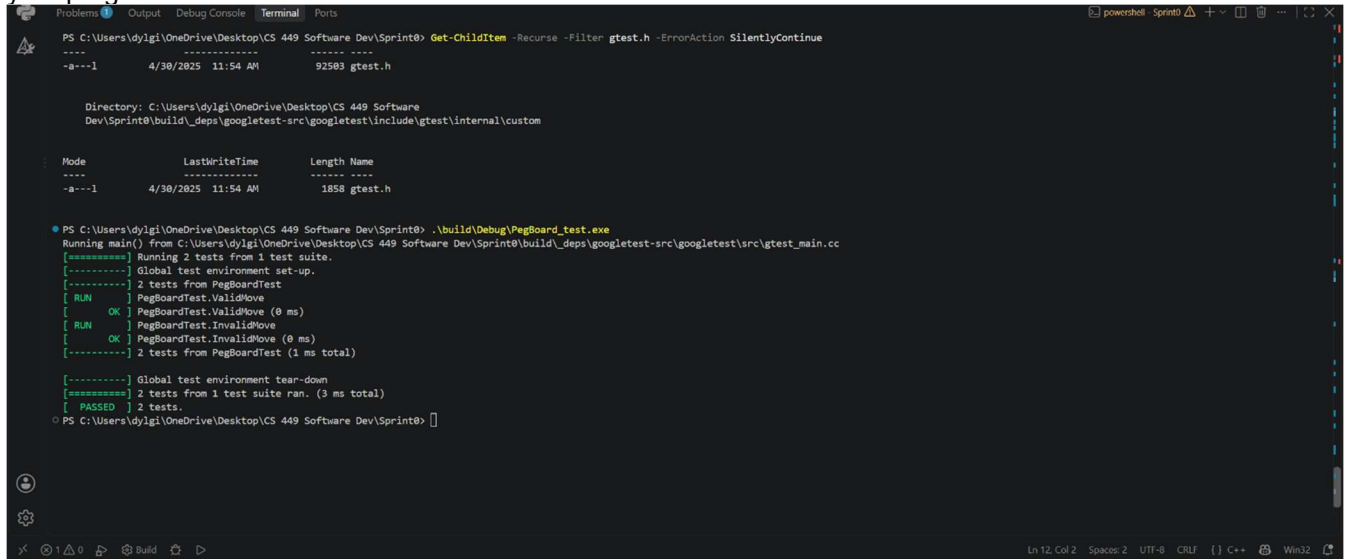
Topic	Decision
Object-oriented programming language (if not using OOP, then justify the use of the language)	C++
GUI library (strongly encouraged)	QT
IDE (Integrated Development Environment)	Visual Code Studio
xUnit framework (e.g., JUnit for Java)	GTest
Programming style guide (must read it carefully)	Google C++ Style Guide
Project hosting site	Github.com
Other decisions if applicable	N/A

Sample programming style guides:

- Google Java Style Guide: <https://google.github.io/styleguide/javaguide.html>
- Google C++ Style Guide: <https://google.github.io/styleguide/cppguide.html>
- Google Python Style Guide: <https://google.github.io/styleguide/pyguide.html>

2. Unit testing (4 points)

Find a tutorial on the unit test framework you have chosen and write at least two xUnit tests of a program you have written or found elsewhere. Attach here (1) the screenshot of your program execution and (2) the source code of your program.



```
PS C:\Users\dy1gi\OneDrive\Desktop\CS 449 Software Dev\Sprint0> Get-ChildItem -Recurse -Filter gtest.h -ErrorAction SilentlyContinue
-----
-a---1      4/30/2025  11:54 AM           92503 gtest.h

Directory: C:\Users\dy1gi\OneDrive\Desktop\CS 449 Software Dev\Sprint0\build_deps\googletest-src\googletest\include\gtest\internal\custom

Mode                LastWriteTime         Length Name
----                -
-a---1      4/30/2025  11:54 AM           1858 gtest.h

PS C:\Users\dy1gi\OneDrive\Desktop\CS 449 Software Dev\Sprint0> .\build\Debug\PegBoard_test.exe
Running main() from C:\Users\dy1gi\OneDrive\Desktop\CS 449 Software Dev\Sprint0\build_deps\googletest-src\googletest\src\gtest_main.cc
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from PegBoardTest
[ RUN      ] PegBoardTest.ValidMove
[ OK       ] PegBoardTest.ValidMove (0 ms)
[ RUN      ] PegBoardTest.InvalidMove
[ OK       ] PegBoardTest.InvalidMove (0 ms)
[-----] 2 tests from PegBoardTest (1 ms total)
[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (3 ms total)
[ PASSED  ] 2 tests.
PS C:\Users\dy1gi\OneDrive\Desktop\CS 449 Software Dev\Sprint0>
```

Code

Header

```
#ifndef PEG_BOARD_H_
#define PEG_BOARD_H_
```

```
class PegBoard {
public:
```

```
    // Determines whether a move is valid.
```

```
    // start_peg: True if there is a peg in the starting position.
    // middle_peg: True if there is a peg in the middle position.
    // end_empty: True if the ending position is empty.
```

```
    // A move is valid only when all three conditions are true:
```

```
    // 1. There is a peg at the starting position.
    // 2. There is a peg at the middle position.
    // 3. The ending position is empty.
```

```
    bool IsValidMove(bool start_peg, bool middle_peg, bool end_empty) {
        return start_peg && middle_peg && end_empty;
    }
```

```
    // Counts the number of pegs currently on the board.
```

```
    // board: An array where each element represents a position on the board.
    // true means that position contains a peg.
    // false means that position is empty.
```

```
    // size: The number of positions/elements in the board array.
```

```
    // Returns the total number of positions that contain a peg.
    int CountPegs(const bool board[], int size) {
```

```

// Start the peg count at zero.
int count = 0;

// Go through every position in the board array.
for (int i = 0; i < size; ++i) {

    // If the current position contains a peg, increase the count.
    if (board[i]) {
        ++count;
    }
}

// Return the total number of pegs found.
return count;
}
};

```

```

#endif

```

PegBoard_test

```

#include "PegBoard.h"
// Includes the GTest framework, which provides the TEST and EXPECT_TRUE/EXPECT_FALSE functions used
below.
#include <gtest/gtest.h>

```

```

// Tests the IsValidMove function when all conditions for a valid move are true.
TEST(PegBoardTest, ValidMove) {

```

```

    PegBoard board;

```

```

    // Checks that IsValidMove returns true when there is a peg at the starting position, a peg in the middle, and the
    ending position is empty.

```

```

    // EXPECT_TRUE passes when the expression evaluates to true.
    EXPECT_TRUE(board.IsValidMove(true, true, true));
}

```

```

// Tests the IsValidMove function when the ending position is not empty.
TEST(PegBoardTest, InvalidMove) {

```

```

    PegBoard board;

```

```

    // Checks that IsValidMove returns false when the ending position is not empty.

```

```

    // EXPECT_FALSE passes when the expression evaluates to false.
    EXPECT_FALSE(board.IsValidMove(true, true, false));
}

```

CMake.txt

```

# Specifies the minimum version of CMake required to build the project.
cmake_minimum_required(VERSION 3.14)

```

```

# Gives the project the name "PegSolitaireTests".
project(PegSolitaireTests)

```

```

# Tells CMake to use the C++17 standard when compiling the project.
set(CMAKE_CXX_STANDARD 17)

# Requires the compiler to support the C++17 standard.
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Generates a compile_commands.json file containing the compiler commands used to build the project.
# Here for debugging purposes while I was getting this to work.
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# Loads CMake's FetchContent module, which allows external libraries to be downloaded and included
automatically.
include(FetchContent)

# Specifies GoogleTest as an external dependency.
FetchContent_Declare(
    googletest

    # Downloads version 1.17.0 of GoogleTest from its GitHub repository.
    URL https://github.com/google/googletest/archive/refs/tags/v1.17.0.zip

    # Uses the time that the files are extracted rather than the timestamps stored in the downloaded archive.
    DOWNLOAD_EXTRACT_TIMESTAMP TRUE
)

# Tells GoogleTest to use the same C runtime library settings as the rest of the project.
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)

# Downloads GoogleTest if necessary and makes it available to the project.
FetchContent_MakeAvailable(googletest)

# Enables CMake's testing functionality.
enable_testing()

# Creates an executable named PegBoard_test.

# PegBoard_test.cpp contains the GoogleTest unit tests.
add_executable(
    PegBoard_test
    PegBoard_test.cpp
)

# GTest::gtest_main provides the main() function needed to start the GoogleTest test program.
target_link_libraries(
    PegBoard_test
    GTest::gtest_main
)

# Loads CMake's GoogleTest integration functionality.
include(GoogleTest)

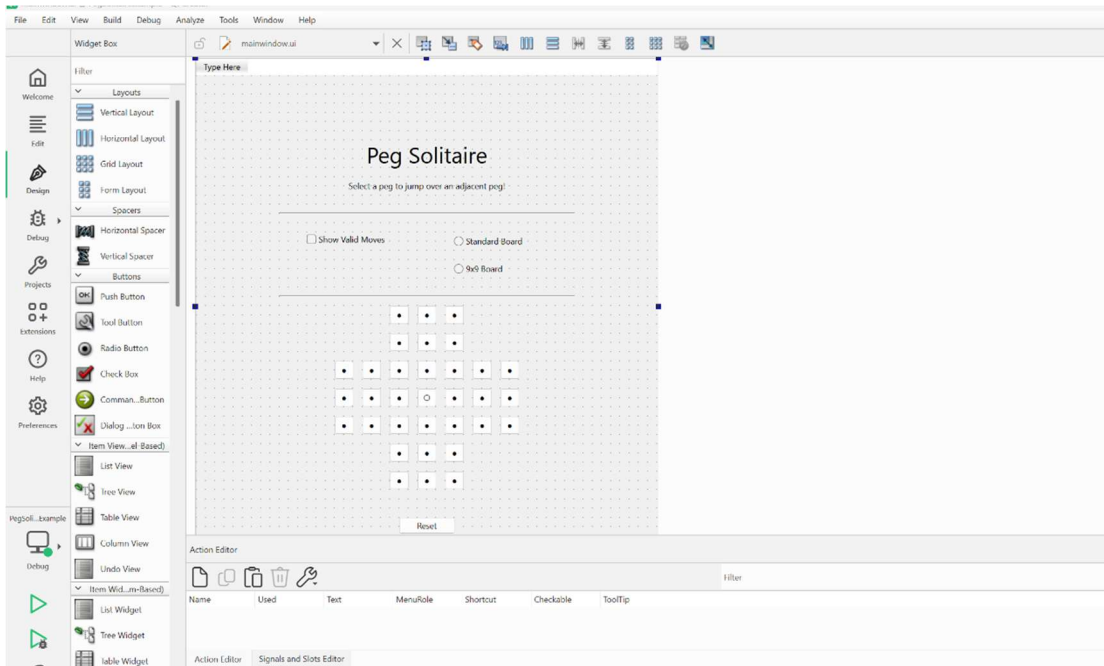
# Automatically discovers the individual GoogleTest tests in PegBoard_test and makes them available to CTest.
gtest_discover_tests(PegBoard_test)

```

3. GUI programming (4 points)

Write a GUI program in the language you have chosen for your Solitaire project. The GUI of your program must include text, lines, a check box, and radio buttons. In this assignment, any GUI program of your own work is acceptable.

Attach here (1) the screenshot of your program execution and (2) the source code of your program.



Source code is in github